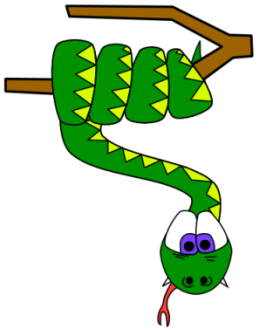
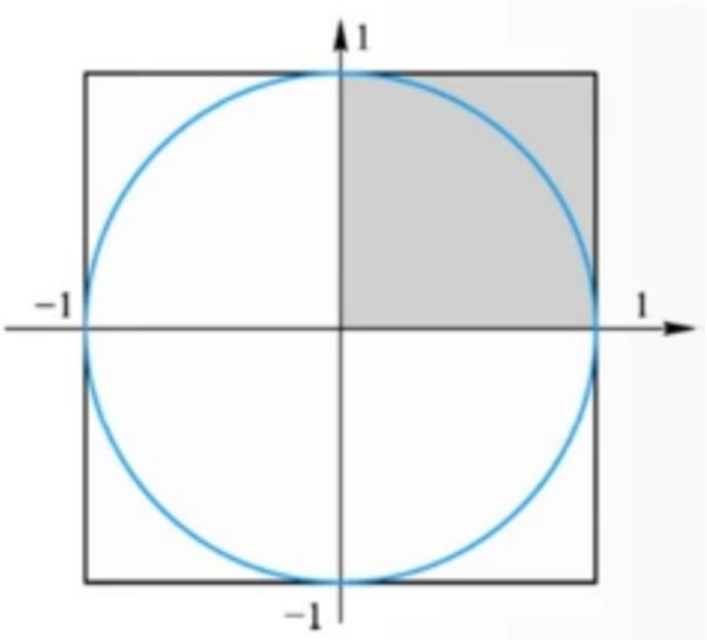


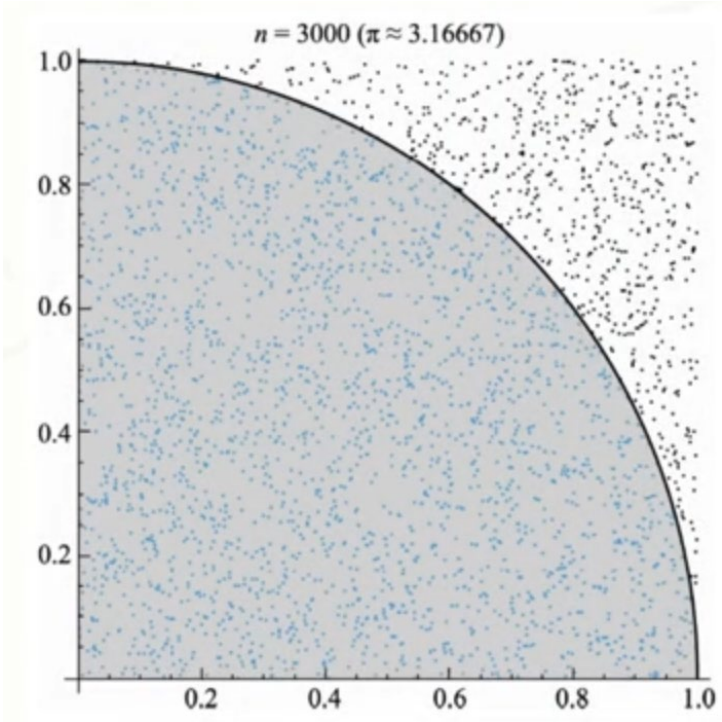
程序流程控制+Turtle简介

邓擎琼

北京师范大学

任务-用蒙特卡罗方法计算圆周率

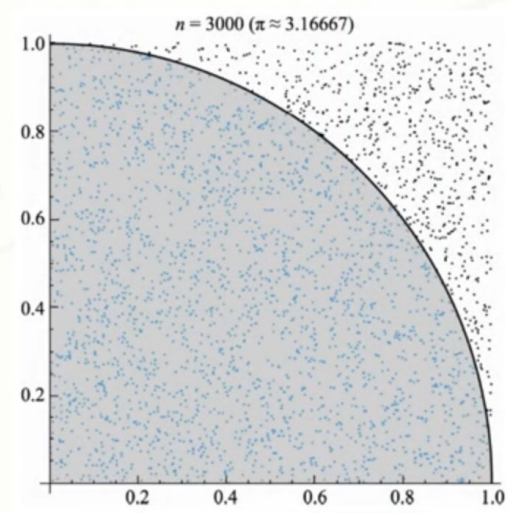




圆和正方形的面积之比是π/4

python实现

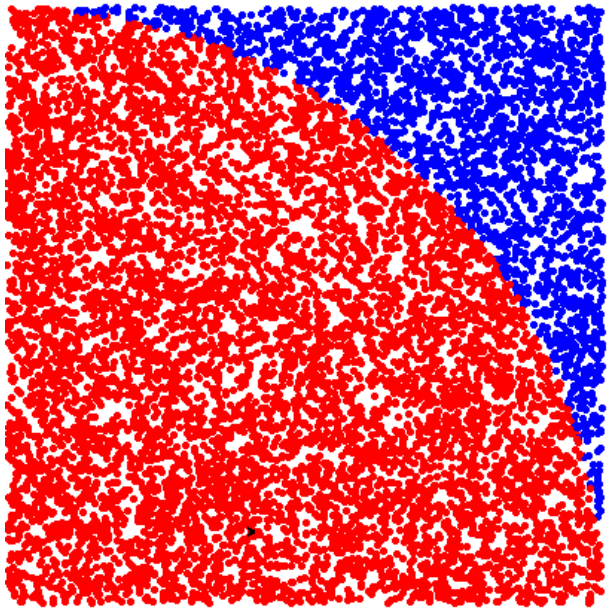
```
from random import random
dots = 1000*1000
ins = 0
for i in range(dots):
    x, y = random(), random()
    dis = pow(x**2+y**2, 0.5)
    if dis <= 1:
        ins += 1
pi = 4*ins/dots
print('圆周率pi={}'.format(pi))
```



圆周率pi=3.140504

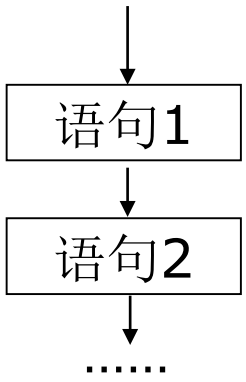
对实现过程进行可视化

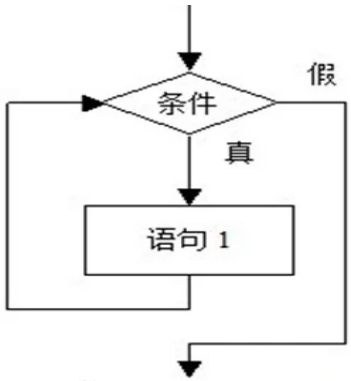
```
import turtle
from random import random
turtle.pensize(2)
dots=100*100
ins=0
for i in range(dots):
    x,y=random(),random()
    dis=pow(x**2+y**2,0.5)
    turtle.penup()
    turtle.goto(400*x,400*y)
    turtle.pendown()
    if dis<=1:
        ins+=1
        turtle.pencolor('red')
    else:
        turtle.pencolor('blue')
    turtle.dot()
pi = 4*ins/dots
print('圆周率pi={}'.format(pi))
turtle.done()
```

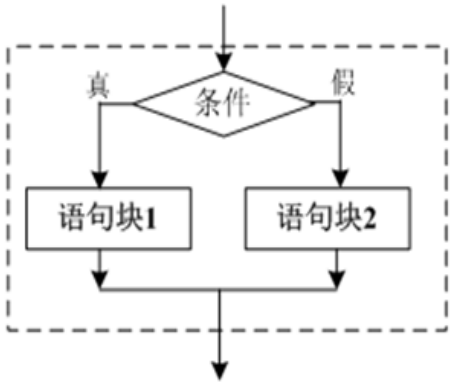


程序流程

- 程序流程：程序步骤
- 三种结构：
  1. **顺序结构**：按照语句的先后顺序依次执行
  2. **分支结构**：根据条件选择不同的语句执行
  3. **循环结构**：语句重复执行





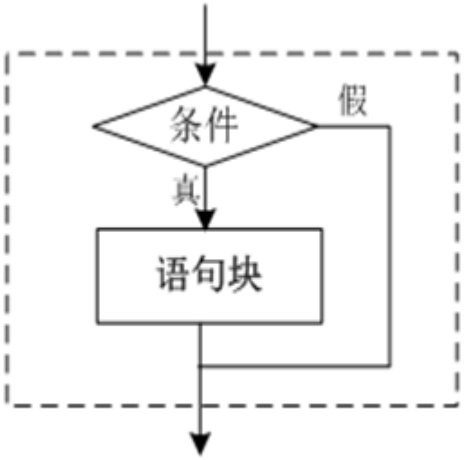


- 所有的程序都可以由顺序、分支和循环结构构成。

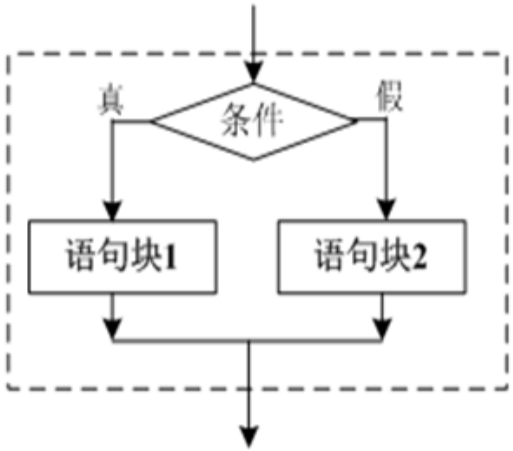
以做菜为例：先放油，然后放香料爆香，之后下菜，反复翻炒2分钟，加盐，如果喜欢甜就加点糖，如果喜欢辣就加点辣椒，...

- 好的流程：结构清晰、易理解、易验证、易维护。

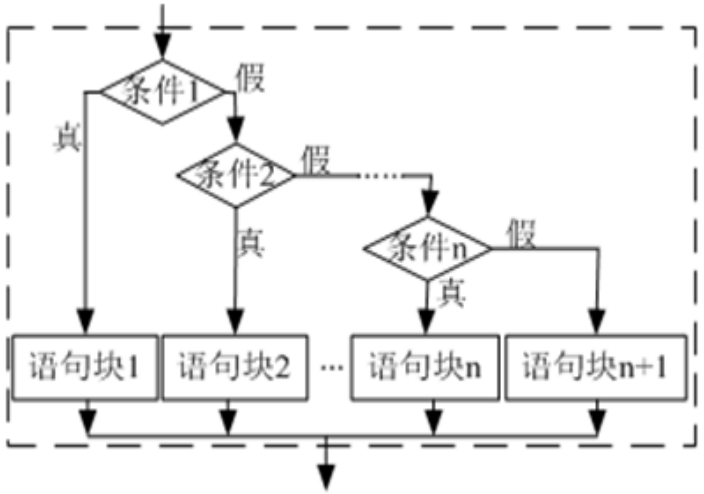
分支结构



单分支



双分支



多分支

```
if 条件表达式:
    语句块
```

```
if 条件表达式:
    语句块1
else:
    语句块2
```

```
if 条件表达式1:
    语句块1
elif 条件表达式2:
    语句块2
…
elif 条件表达式n:
    语句块n
```

缩进：4个空格

可选

```
else:
    其他情形语句块
```

条件表达式

- 任何非零数字或非空对象都为真
- 数字0，空对象以及特殊对象None都被认作是假
- 关系表达式：==, !=, >, >=, <, <=, is, is not等
- 逻辑表达式：and, or, not

例1. 判断数字i是否是偶数：　　i%2==0

例2. 输入三条边长a,b,c，判断是否构成三角形：

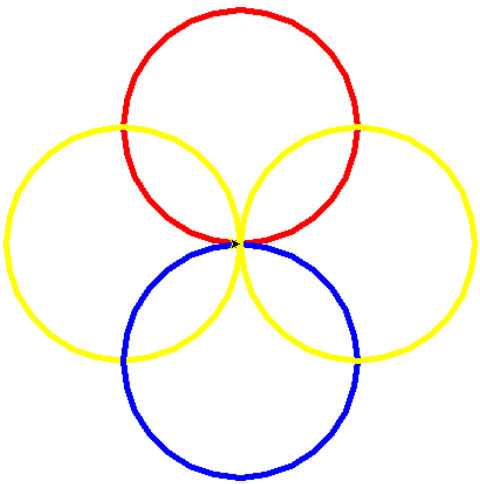
a+b>c and a+c>b and b+c>a

单分支示例

- 输入两个整数a和b，比较大小，使得a>=b

```
a=int(input('请输入第1个整数: '))
b=int(input('请输入第2个整数: '))
if a<b:
    a,b=b,a
print('a=%d,b=%d'%(a,b))
```

双分支示例



```
#绘制一朵彩色花
import turtle

t = turtle.Pen()
t.pensize(5)
colors = ['red', 'yellow','blue']

n=turtle.numinput('输入', '花瓣数:', 3, 3, 100)

for i in range(int(n)):
    if i==n-1 and n%3==1:
        t.pencolor(colors[1])
    else:
        t.pencolor(colors[i%3])
    t.circle(100)
    t.left(360/n)

turtle.done()
```

双分支示例2

- 计算分段函数 $y=\begin{cases}\sin x+2\sqrt{x+e^4}-(x+1)^3 & x\geq 0\\ \ln(-5x)-\dfrac{|x^2-8x|}{7x}+e & x<0\end{cases}$

```
import math
x=float(input('请输入x的值：'))
if x>=0:
    y=math.sin(x)+2*math.sqrt(x+math.exp(4))-math.pow(x+1,3)
else:
    y=math.log(-5*x)-math.fabs(x*x-8*x)/(7*x)+math.e
print('当x={0}时，y={1}'.format(x,y))
```

- Python提供条件赋值语句
  - 格式：条件为真时的值 if 条件表达式 else 条件为假时的值
  - 例如，如果x>=0，则y=x，否则y=0，语句为：

    y=x if x>=0 else 0

条件赋值语句示例

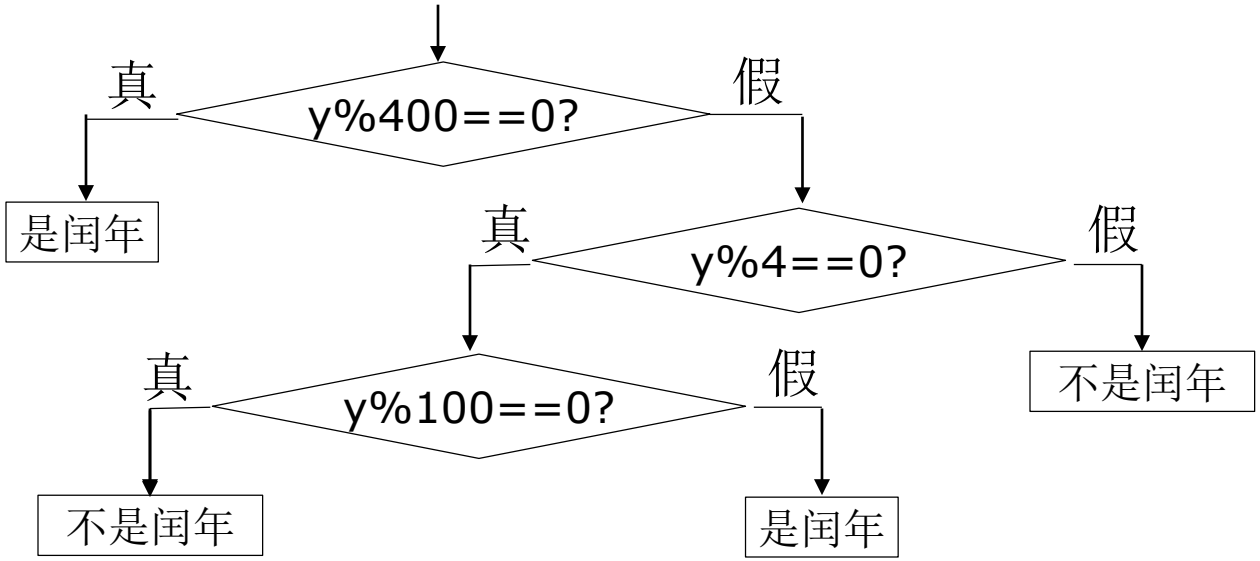
- 计算分段函数 $y=\begin{cases}\sin x+2\sqrt{x+e^4}-(x+1)^3 & x\geq 0\\ \ln(-5x)-\dfrac{|x^2-8x|}{7x}+e & x<0\end{cases}$

续行符

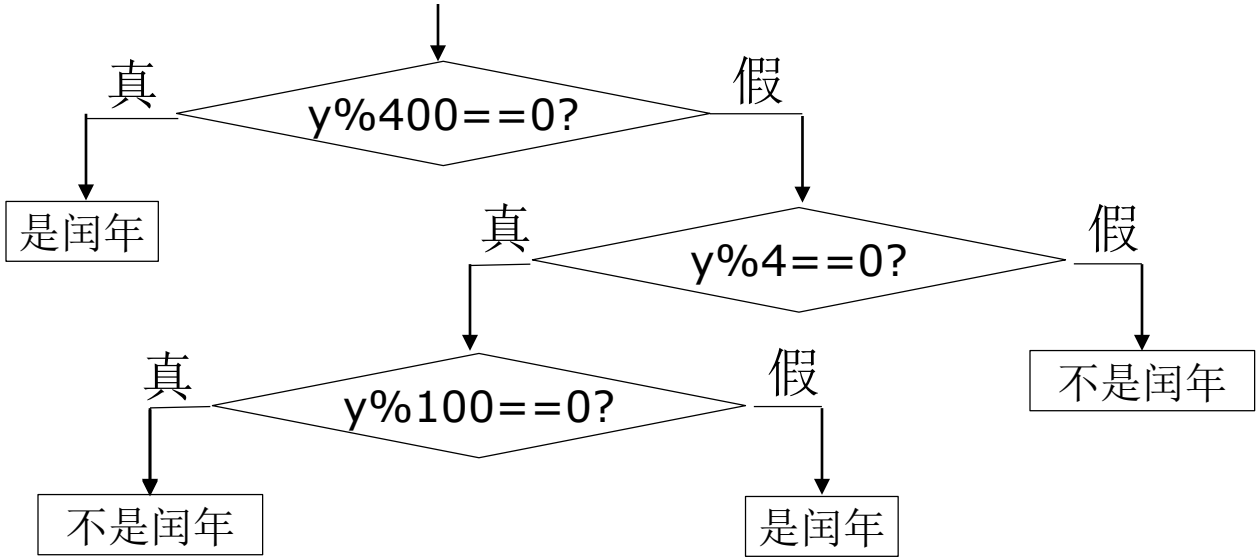
```
import math
x=float(input('请输入x的值：'))
y=math.sin(x)+2*math.sqrt(x+math.exp(4))-math.pow(x+1,3) \
if x>=0 else y=math.log(-5*x)-math.fabs(x*x-8*x)/(7*x)+math.e
print('当x={0}时，y={1}'.format(x,y))
```

分支嵌套

- 分支嵌套：分支里又包含分支
- 例如：判断某一年是否为闰年。闰年的条件是：年份能被400整除，或者能被4整除但不能被100整除。



分支嵌套示例代码

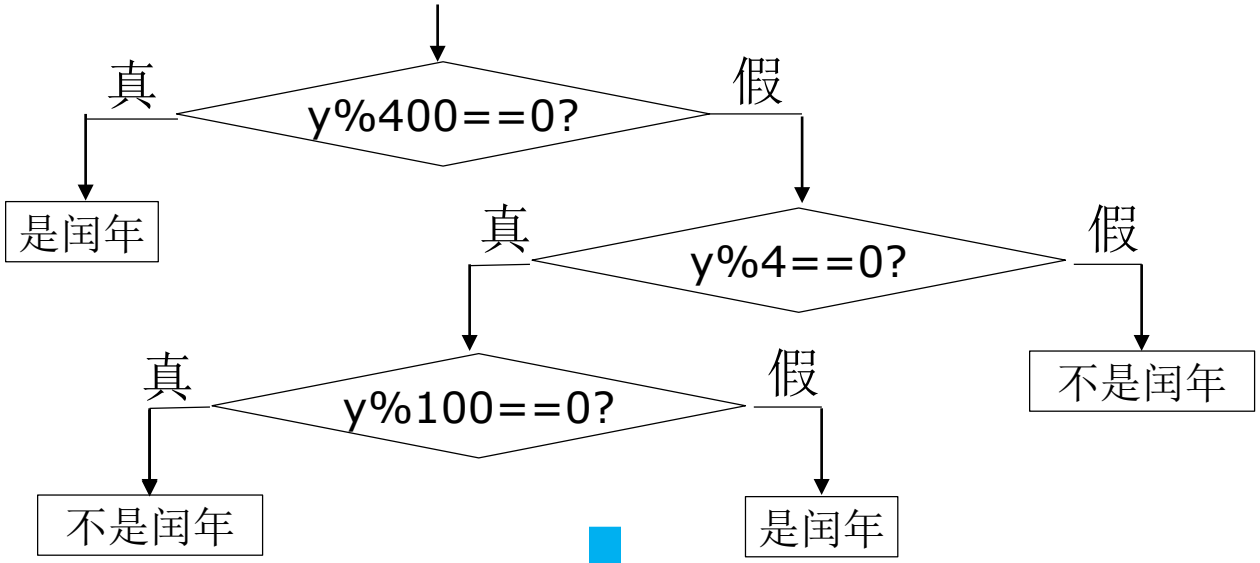


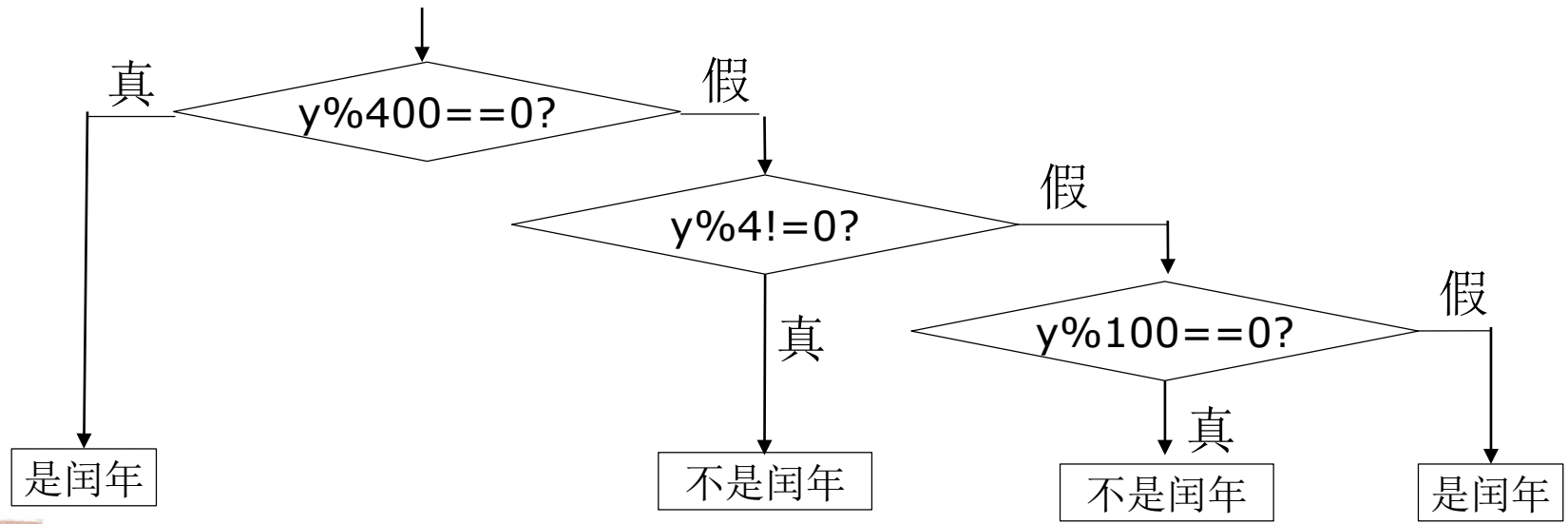

```
y=int(input('请输入年份: '))
if y%400==0: print('是闰年')
else:
    if y%4==0:
        if y%100==0: print('不是闰年')
        else: print('是闰年')
    else: print('不是闰年')
```

多重嵌套不好

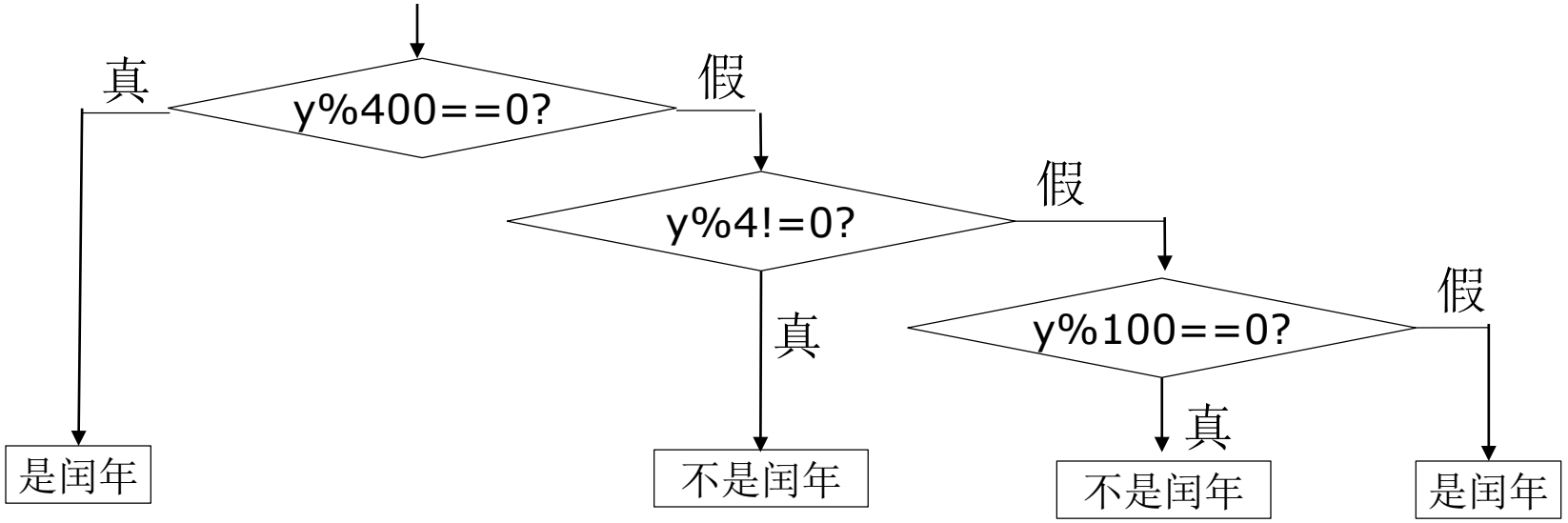
- 难读
- 代码松散
- 向右缩进占空间

多分支结构


y%400==0?
真
假
是闰年
y%4==0?
真
假
不是闰年
y%100==0?
真
假
不是闰年
是闰年


y%400==0?
真
假
是闰年
y%4!=0?
假
真
不是闰年
y%100==0?
假
真
不是闰年
是闰年

多分支示例




```
y=int(input('请输入年份: '))
if y%400==0: print('是闰年')
elif y%4!=0: print('不是闰年')
elif y%100==0: print('不是闰年')
else: print('是闰年')
```

循环控制结构

- 循环结构：重复执行一条或多条语句
- Python 提供两种循环结构：
  - for 循环
  - while 循环

for 循环

- 语法：

  for 变量 in 可迭代对象集合:

  循环体语句/语句块

- 语义:令变量取遍集合中的每一个元素，并对变量所取的每个元素值执行一遍循环体语句/语句块。

- 例如:

```
>>> for i in [1,2,3,4]:
        print(i, i**2, i**3)
```

```
1 1 1
2 4 8
3 9 27
4 16 64
```

用for处理各种序列数据示例

- 字符串

```
>>> for c in 'hello':
        print(c)

h
e
l
l
o
```

- 元组

```
>>> for i in(1,2,3):
        print(i)

1
2
3
```

- 嵌套序列:如元组的列表

```
>>> for t in [(1,2),(3,4),(5,6)]:
        print(t,t[0],t[1])

(1, 2) 1 2
(3, 4) 3 4
(5, 6) 5 6
```

- 多个循环控制变量构成元组

```
>>> for x,y in [(1,2),(3,4),(5,6)]:
        print(x+y)

3
7
11
```

range对象

- 显示1~10000:
  - 用[1,2,...,10000]显然不合适，可以用range对象。

- range格式： range(start, stop [,step])　range(stop)
  - range返回的数值序列从start开始，到stop结束（不包含stop）。另外如果指定了可选的步长step，则序列按步长增长。
  - 注意：所有参数均为整数！

- 例子：

range(1,11)　→ 1 2 3 4 5 6 7 8 9 10
range(1,11,3)　→ 1 4 7 10
range(10,0,-1)　→ 10 9 8 7 6 5 4 3 2 1
range(11)　→ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

range对象2

- 使用示例：求1~100中所有奇数和偶数的和

```
sum_odd=0; sum_even=0
for i in range(1, 101):
    if i%2==0:
        sum_even +=i
    else:
        sum_odd +=i
print('1~100中所有奇数的和是{}，\
所有偶数的和是{}'.format(sum_odd,sum_even))
```

可迭代

```
>>> print(range(1,101))
range(1, 101)
```

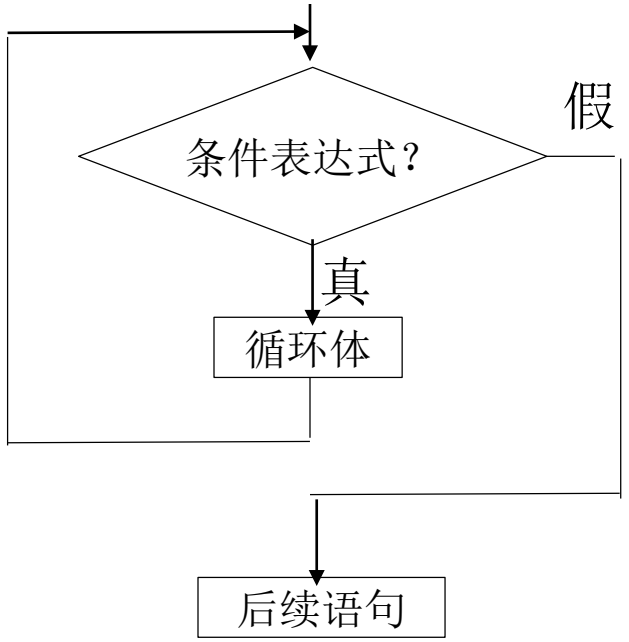
惰性的，节省内存

while循环

- for主要用于确定次数的循环
- 不确定次数的循环：while

  while 条件表达式:

  循环体语句/语句块



☆ 语义：当条件表达式计算结果为真时，执行一遍循环体，执行完毕控制转回while语句开始处重新测试条件表达式；当条件表达式计算结果为False，控制转向下一条语句。

while循环示例

- 用如下近似公式求自然对数的底数e的值，直到最后一项的绝对值小于10^{-6}为止。

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \ldots + \frac{1}{n!}$$

```
i=1; e=1; t=1
while 1/t>=pow(10,-6):
    t*=i
    e+=1/t
    i+=1
print('e=', e)
```

运行结果： e= 2.7182818011463845

循环非正常中断:break

- 立即结束break所在循环体语句

```
counts = {'小龙女':'111', '杨过': '123', '黄蓉':'456' }

while True:
    username = input('请输入账号：')
    password = input('请输入密码：')
    if username in counts and password == counts[username]:
        print('登录成功！')
        break
    print('输入错误，请重新输入！')
```

```
请输入账号：小龙女

请输入密码：123
输入错误，请重新输入！

请输入账号：小龙女

请输入密码：111
登录成功！
```

循环非正常中断:continue

- 中止本轮循环，控制转移到所处循环语句的开头“继续”下一轮循环。

- 例:对列表中的奇数求和

```
a = [23,28,39,44,50,67,99]
sum = 0
for i in a:
    if i%2 == 0:
        continue
    sum = sum + i
print(sum)
```

else子句

- for、while语句可以附带一个else子句（可选）。

- 语法：

```
for 变量 in 序列:
    循环体语句（块）1
else:
    语句（块）2
```

```
while(条件表达式):
    循环体语句（块）1
else:
    语句（块）2
```

- 如果for 、while语句没有被break语句中止，就会执行else子句，否则不会。

else 子句示例

- 账号密码登录，3次输入错误则24小时之后再尝试。

```
counts = {'小龙女':'111', '杨过': '123', '黄蓉':'456' }
i = 1
while i<=3:
    username = input('请输入账号: ')
    password = input('请输入密码: ')
    if username in counts and password == counts[username]:
        print('登录成功! ')
        break
    print('输入错误，你还有{}次机会! '.format(3-i))
    i += 1
else:
    print('请24小时后再试! ')
```

请输入账号：小龙女

请输入密码：111
登录成功!

请输入账号：小丽

请输入密码：111
输入错误，你还有2次机会!

请输入账号：小爱

请输入密码：111
输入错误，你还有1次机会!

请输入账号：天天

请输入密码：111
输入错误，你还有0次机会!
请24小时后再试!

嵌套循环示例

- 打印九九乘法表

```
for i in range(1,10):
    for j in range(1,i+1):
        print('{}*{}={}'.format(j,i,i*j), end='\t')
    print()
```

```
1*1=1
1*2=2   2*2=4
1*3=3   2*3=6   3*3=9
1*4=4   2*4=8   3*4=12  4*4=16
1*5=5   2*5=10  3*5=15  4*5=20  5*5=25
1*6=6   2*6=12  3*6=18  4*6=24  5*6=30  6*6=36
1*7=7   2*7=14  3*7=21  4*7=28  5*7=35  6*7=42  7*7=49
1*8=8   2*8=16  3*8=24  4*8=32  5*8=40  6*8=48  7*8=56  8*8=64
1*9=9   2*9=18  3*9=27  4*9=36  5*9=45  6*9=54  7*9=63  8*9=72  9*9=81
```

Turtle简介

- 海龟绘图

  https://docs.python.org/3/library/turtle.html

  中文网站：https://docs.python.org/zh-cn/3/library/turtle.html

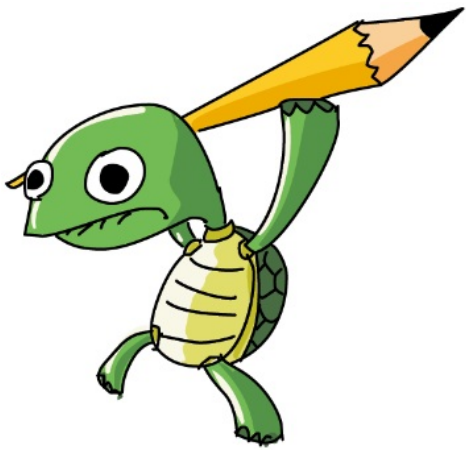
- 导入：

常用方式1：

```
>>> import turtle
>>> turtle.forward(100)
```

常用方式2：

```
>>> from turtle import *
>>> forward(100)
```

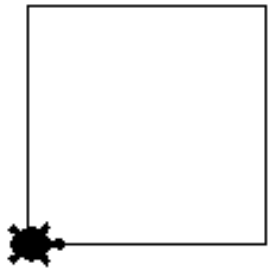


常用函数/方法

- 海龟运动：
  - forward()/fd(), backward()/back()/bk(),
  - right()/rt(), left()/lt(), setheading()/seth()
  - goto(), home()
  - circle(), dot()
  - undo(), speed(), ……
- 画笔：
  - pendown()/pd()/down(), penup()/pu()/up()
  - pensize()/width()
  - color(), pencolor(), fillcolor()
  - begin_fill(), end_fill()
  - clear(), reset(), ……
- 画布：
  - screensize(), setup()
  - bgcolor(), colormode()
  - clear(), reset()
  - mainloop()/done(), ……

示例1-绘制正方形

```
>>> from turtle import *
>>> reset()
>>> shape('turtle')
>>> forward(100)
>>> left(90)
>>> forward(100)
>>> left(90)
>>> forward(100)
>>> left(90)
>>> forward(100)
>>> left(90)
```



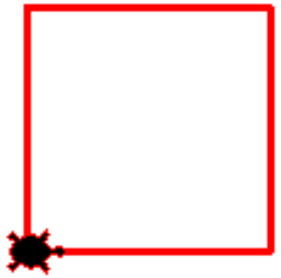
- 用循环实现：

```
>>> for i in range(4):
        forward(100)
        left(90)
```

示例1续-绘制正方形

- 设定笔的宽度和颜色：

```
>>> pensize(3)
>>> pencolor('red')
>>> for i in range(4):
        forward(100)
        left(90)
```

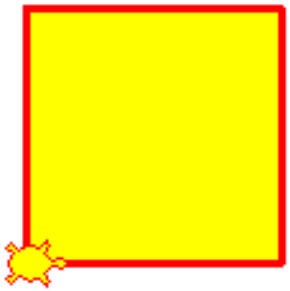


- 填充颜色：

```
>>> pensize(3)
>>> pencolor('red')
>>> fillcolor('yellow')
>>> begin_fill()
>>> for i in range(4):
        fd(100)
        lt(90)
```

等同于：

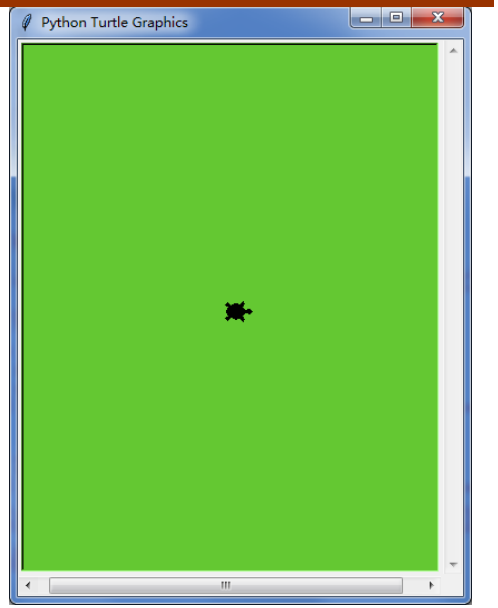
```
color('red', 'yellow')
```



```
>>> end_fill()
```

示例1续-绘制正方形

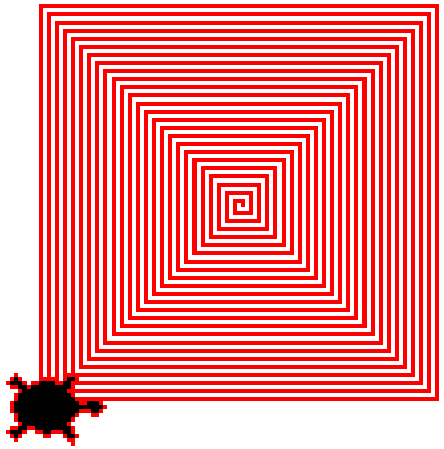
- 设定背景颜色等




```
>>> setup(400,500)
>>> bgcolor('black')
>>> bgcolor(100,200,50)
Traceback (most recent call last):
  File "<pyshell#47>", line 1, in <module>
    bgcolor(100,200,50)
  File "<string>", line 8, in bgcolor
  File "C:\Python\Python36\lib\turtle.py", line 1237, in bgcolor
    color = self._colorstr(args)
  File "C:\Python\Python36\lib\turtle.py", line 1166, in _colorstr
    raise TurtleGraphicsError("bad color sequence: %s" % str(color))
turtle.TurtleGraphicsError: bad color sequence: (100, 200, 50)
>>> colormode()
1.0
>>> colormode(255)
>>> bgcolor(100,200,50)
```

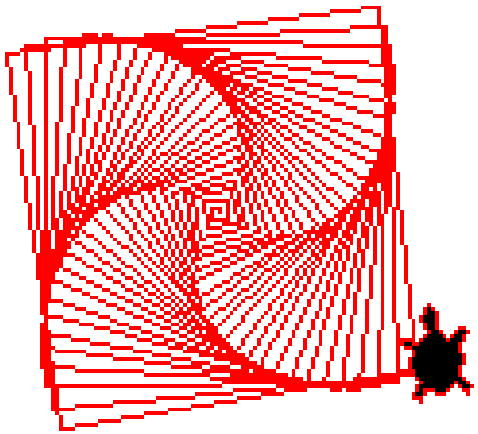
示例2-正方形螺旋线

```
>>> pencolor('red')
>>> for i in range(100):
        forward(i)
        left(90)
```



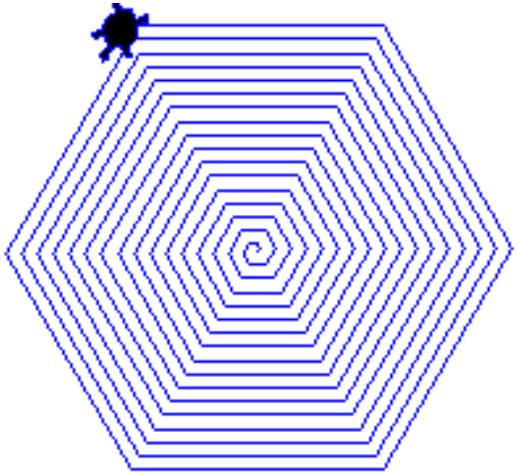
- 旋转的螺旋线

```
>>> pencolor('red')
>>> for i in range(100):
        forward(i)
        left(91)
```



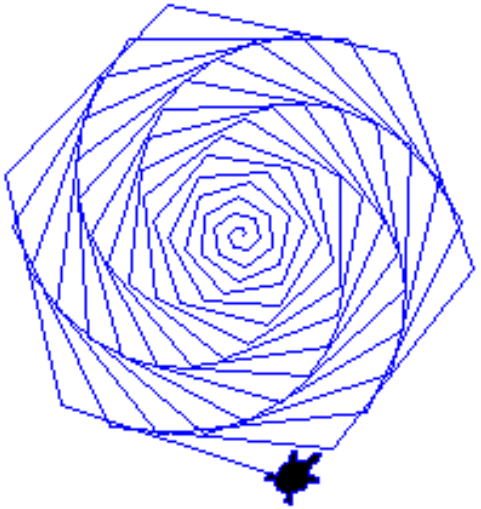
示例3-六边形螺旋线

```
>>> pencolor('blue')
>>> for i in range(100):
        fd(i)
        lt(60)
```



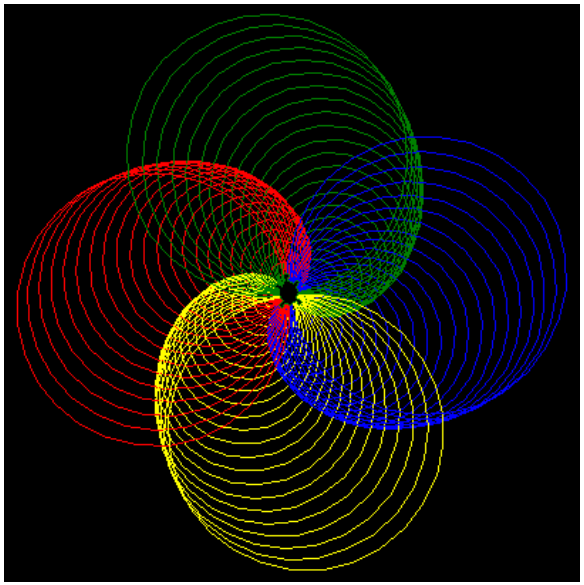
- 旋转的螺旋线

```
>>> pencolor('blue')
>>> for i in range(100):
        fd(i)
        lt(58)
```



示例4-圆形螺旋线

```
>>> bgcolor('black')
>>> colors=['red','yellow','blue','green']
>>> for i in range(100):
        pencolor(colors[i%4])
        circle(i)
        left(91)
```



示例5-Python小蛇

```
import turtle
turtle.clear()
turtle.setup(800, 1000, 0, 0)
turtle.penup()
turtle.fd(-250)
turtle.pendown()
turtle.pensize(25)
turtle.pencolor('green')
turtle.seth(-40)
for i in range(4):
    turtle.circle(40, 80)
    turtle.circle(-40, 80)
turtle.circle(40, 80/2)
turtle.fd(40)
turtle.circle(16, 180)
turtle.fd(40 * 2/3)
turtle.done()
```

示例6-蒙特卡罗方法求解pi

```
import turtle
from random import random

dots=100*100
ins=0
turtle.speed(0) #最快龟速
turtle.tracer(1000) #每1000个动作更新一下屏幕
turtle.pensize(2)
for i in range(dots):
    x,y=random(),random()
    dis=pow(x**2+y**2,0.5)
    turtle.penup()
    turtle.goto(500*x,500*y)
    turtle.pendown()
    if dis<=1:
        ins+=1
        turtle.pencolor('red')
    else:
        turtle.pencolor('blue')
    turtle.dot()
pi = 4*ins/dots
turtle.penup()
turtle.goto(150,-50)
turtle.pendown()
turtle.pencolor('black')
turtle.write("圆周率pi={}".format(pi), align="left",\
             font=("Arial", 20, "bold"))
turtle.done()
```

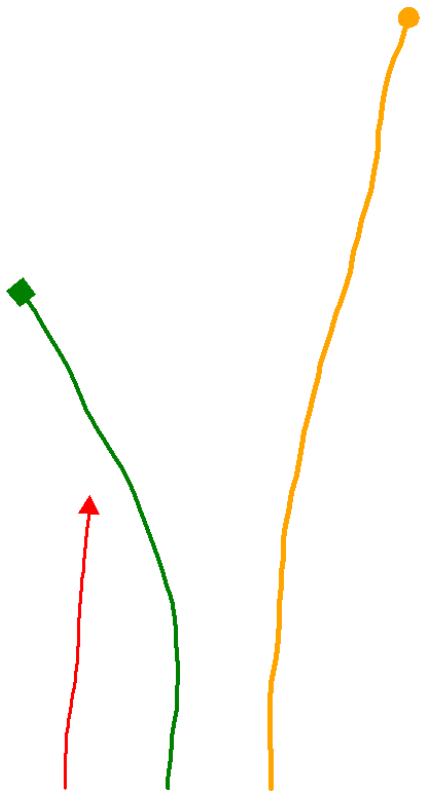
面向对象的绘图

- 要在屏幕上使用多个海龟，必须使用面向对象的绘图方式

```
from turtle import *
one=Turtle(shape='triangle',visible=False)
two=Turtle(shape='square',visible=False)
three=Turtle(shape='circle',visible=False)
one.pensize(3)
two.pensize(4)
three.pensize(5)
one.color('red')
two.color('green')
three.color('orange')
one.penup()
two.penup()
three.penup()
one.goto(-100,-300)
two.goto(0,-300)
three.goto(100,-300)
one.left(90)
two.left(90)
three.left(90)
one.showturtle()
two.showturtle()
three.showturtle()
one.pendown()
two.pendown()
three.pendown()
```

```
import random
for i in range(50):
    f=random.randint(0,10)
    s=random.randint(6,15)
    t=random.randint(10,20)

    fa=random.randint(-3,3)
    sa=random.randint(-5,5)
    ta=random.randint(-7,7)

    one.forward(f)
    two.forward(s)
    three.forward(t)

    one.left(fa)
    two.left(sa)
    three.left(ta)

done()
```

模拟三棵小苗
的生长过程

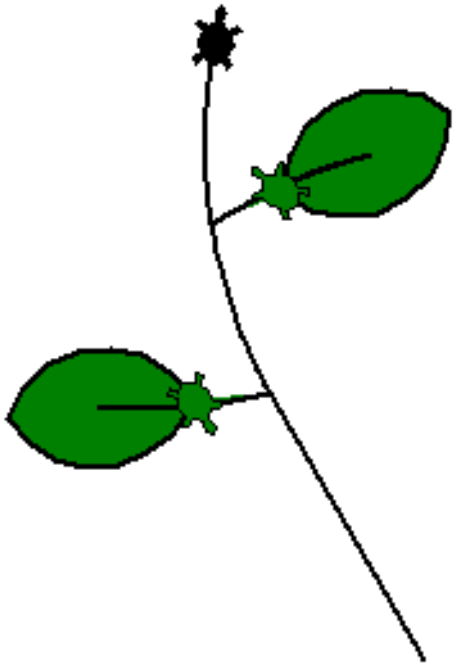


面向对象的绘图续

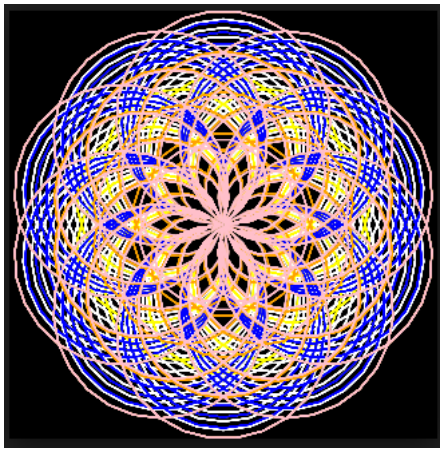
- **clone方法的使用**

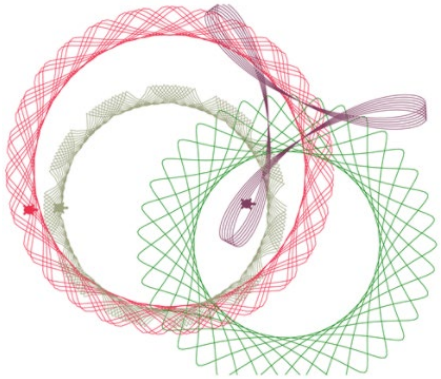
```
from turtle import*
p = Turtle()
p.pensize(2)
p.speed(1)
p.shape('turtle')
p.left(120)
p.fd(120)
q = p.clone()
q.fillcolor('green')
q.begin_fill()
q.left(70)
q.forward(20)
q.circle(-170,16)
q.circle(-170,-12)
q.left(60)
q.circle(-40,125)
q.right(50)
q.circle(-40,130)
q.end_fill()
```

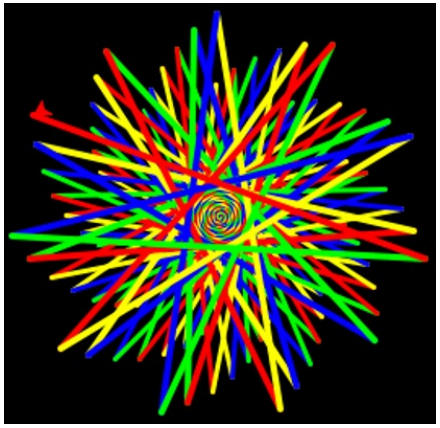
```
for i in range(20):
    p.rt(2)
    p.circle(200,1)
q=p.clone()

q.fillcolor('green')
q.begin_fill()
q.right(70)
q.forward(20)
q.circle(-170,16)
q.circle(-170,-12)

q.left(60)
q.circle(-40,125)
q.right(50)
q.circle(-40,130)
q.end_fill()

for i in range(20):
    p.rt(2)
    p.circle(200,1)
```

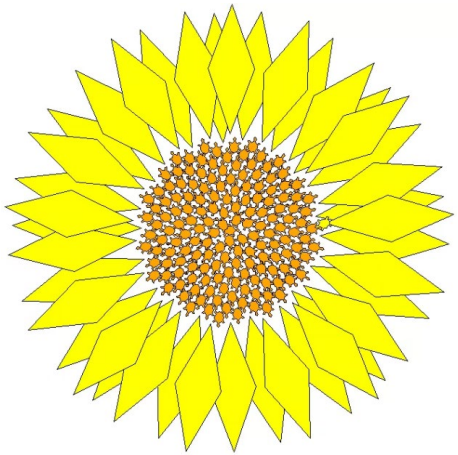


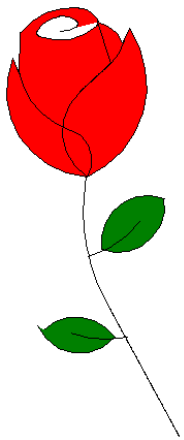
Turtle 绘图欣赏

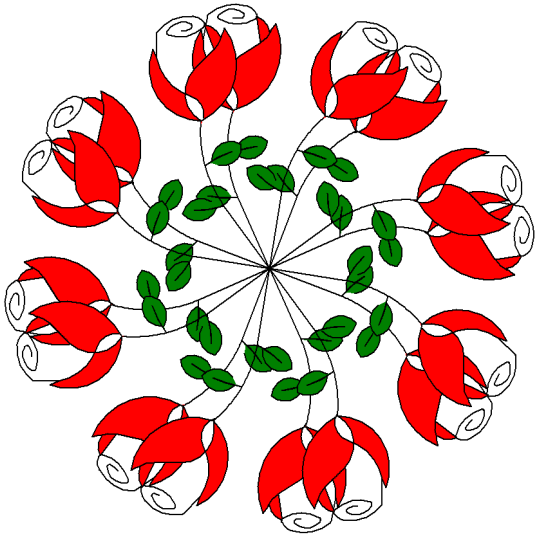


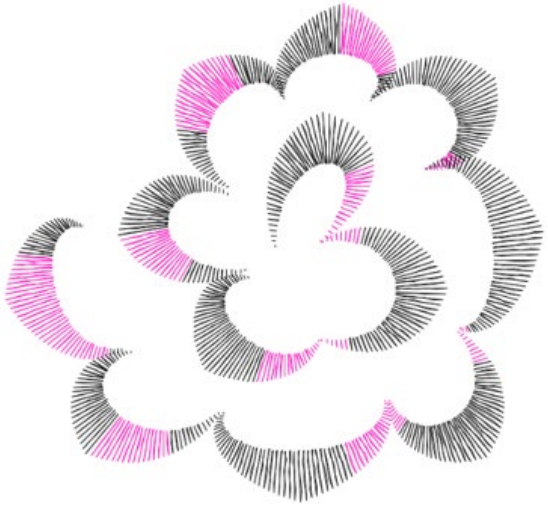


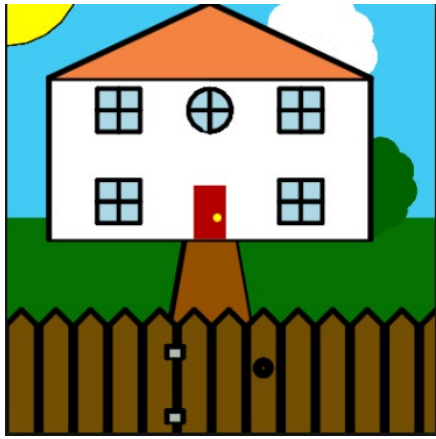




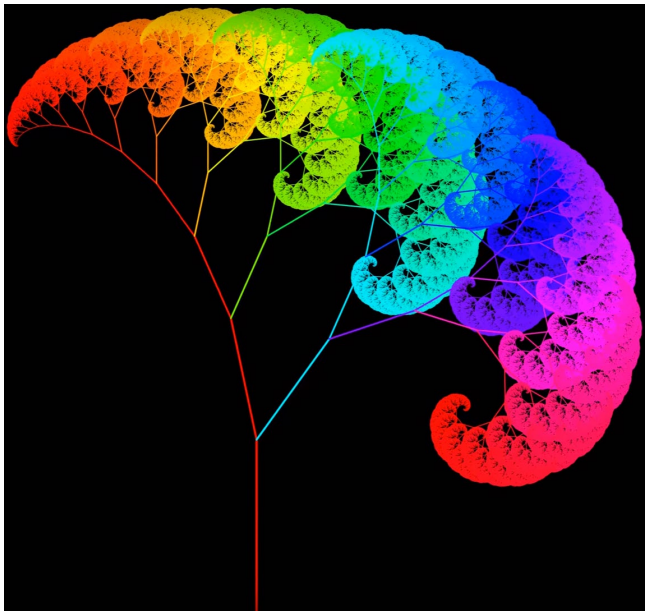


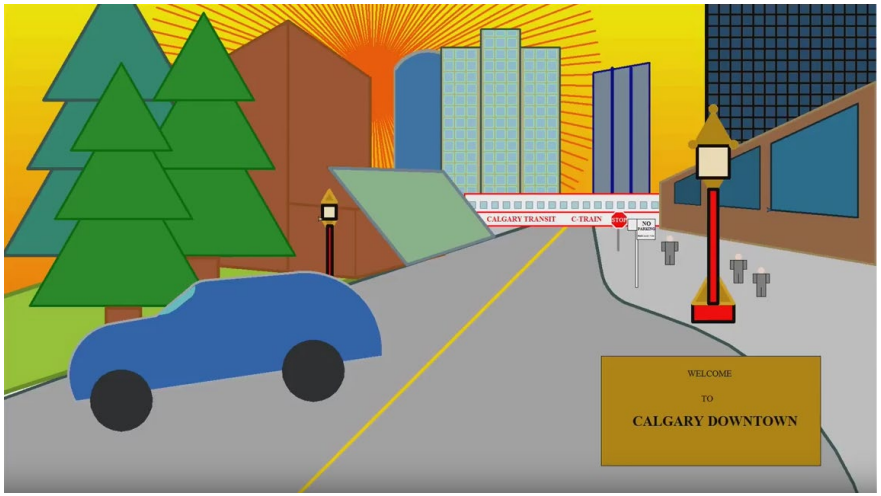


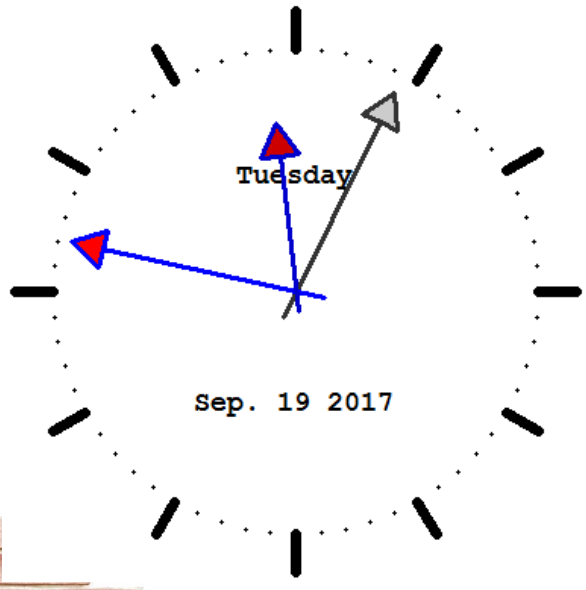


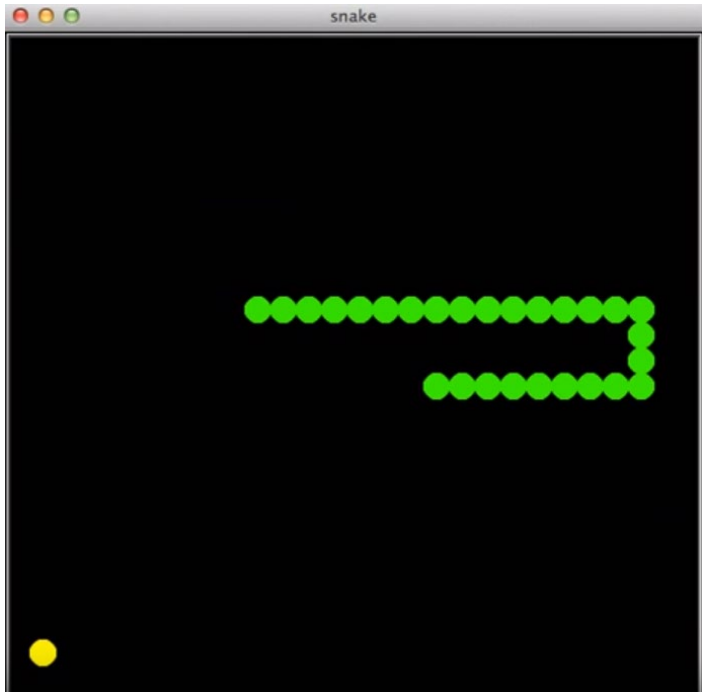


Turtle 绘图欣赏




CALGARY TRANSIT
C-TRAIN
WELCOME
TO
CALGARY DOWNTOWN


Tuesday
Sep. 19 2017


snake